

Advanced Lesson 6 - Exercises:

Replacement Rules and Evaluation

Mikey Ferguson

MATH 7013 - Advanced Mathematical Modeling

1: $x = 2$ followed by an integration in x

```
Clear[x];
```

```
x = 2;
```

$$\int x^2 dx$$

Why it misbehaves. Integrate does not hold its arguments. By the time Integrate sees them, x has already been replaced by 2 in both slots, so the call that actually runs is Integrate[4, 2] - integrate the constant 4 with respect to the number 2. Mathematica reports Integrate::ilim, "Invalid integration variable or limit(s) in 2", and returns the expression unevaluated. The integration variable was destroyed by the assignment before the integral was ever set up.

2: $x := 2$ followed by the same integration

```
Clear[x];
```

```
x := 2;
```

$$\int x^2 dx$$

Identical failure, identical message. Delayed assignment changes when the right-hand side is evaluated, not whether x has a value. Every time x is looked up it still yields 2, so Integrate still receives Integrate[4, 2]. Delayed assignment is not a way to keep a symbol free.

3: A global x colliding with a pattern variable named x

```
Clear[x, f1, f2];
x = 2;
f1[x_] = x^2;
f2[x_] := x^2;

?f1
?f2

{f1[z], f2[z]}

Plot[{f1[z], f2[z]}, {z, 0, 3}, PlotStyle -> {Thickness[0.01`]}]
```

Why it misbehaves. Set holds its left-hand side, so the pattern `f1[x_]` survives intact - the name `x` on the left is a pattern variable and is not touched. The right-hand side is a different matter: it is evaluated immediately, and there `x` means the global `x`, which is 2. So `x^2` evaluates to 4 and the stored rule is `f1[x_] :=> 4`. "`?f1`" confirms it. `f1` returns 4 for every argument, and its curve is a horizontal line.

`f2` uses delayed assignment, so the right-hand side is stored unevaluated as `x^2` with `x` still the pattern variable. "`?f2`" shows `f2[x_] :=> x^2`, `f2[z]` returns `z^2`, and its curve is the parabola.

The general lesson: naming a pattern variable the same as a symbol that already has a global value is safe on the left of a definition and fatal on the right of an immediate one.

4: A replacement rule and an integration in the wrong variable

```
Clear[x, y];
x^2 /. x -> y


$$\int x^2 \, dy$$

```

Why it misbehaves. The replacement rule works fine and returns `y^2` - but `ReplaceAll` does not assign anything. It produces a new expression and throws it away; `x` is still `x` afterwards. So the integral that follows is `Integrate[x^2, y]`: integrating a constant (as far as `y` is concerned) with respect to `y`, which returns `x^2 y`. The mistake is expecting `"/."` to have changed `x` permanently. It never does.

5: Assigning the result of a self-replacement

```
Clear[x, y, z];
```

```
x = x /. x -> y
```

$$\int x^2 dy$$

Here the replacement result is assigned, so x really does become y. The integral is then `Integrate[y^2, y]`, which returns $y^3/3$. This is the behaviour Exercise 4 was expected to have and did not, and the only difference is the assignment.

6: The same shape, with a symbol the rule does not mention

```
Clear[x, y, z];
```

```
x = z /. x -> y
```

$$\int x^2 dy$$

The rule `x -> y` is applied to the expression `z`, which contains no `x`, so nothing is replaced and `z` comes back unchanged. That `z` is then assigned to `x`. The integral is therefore `Integrate[z^2, y]` - a constant with respect to `y` - which returns $y z^2$. The rule was irrelevant; the assignment did all the work.

7: Why two rules in one list do not chain

```
Clear[x, y];
```

```
Sin[x] /. {Sin[x] -> Log[x], x -> 2 Pi y}
```

Why it does not give `Log[2 Pi y]`. `ReplaceAll` scans the expression once, from the top down, and at each position it tries the rules in order and stops at the first that matches. At the top, `Sin[x]` matches the first rule and is replaced by `Log[x]`. Having replaced that subexpression, `ReplaceAll` does not descend into the result - the freshly inserted `Log[x]` is output, not input. So the second rule never gets a chance at the `x` inside it. The answer is `Log[x]`.

Two ways to fix it, both below: apply the rules repeatedly until nothing changes, or write a single rule that does the whole job at once.

```
Sin[x] //. {Sin[x] -> Log[x], x -> 2 Pi y}
```

```
Sin[x] /. Sin[x] → Log[2 π y]
```

ReplaceRepeated ("//.") re-scans until the expression stops changing, so the second rule catches the x that the first rule introduced, and the result is Log[2 Pi y]. The single-rule version reaches the same place without any iteration, and is the safer choice whenever the substitution is known in advance - repeated replacement on a rule set that can regenerate its own left-hand side will loop forever.

8: A rule set carried inside a function definition

```
Clear[f, x, y, z];
f[x_] := x /. {y → Cos[y], z → Sin[z]}

f[x]

f[x + y + z]
```

How it works. The pattern variable x captures whatever argument is supplied, and the body applies the rule list to it. The rules mention y and z only, so the function rewrites any y into Cos[y] and any z into Sin[z] and leaves everything else alone.

f[x] returns x - the argument contains no y and no z, so nothing matches. f[x + y + z] returns x + Cos[y] + Sin[z]. ReplaceAll descends into the sum and treats each summand as a separate position, so both rules fire, each once. Note again that neither rule fires on its own output: the y inside the newly created Cos[y] is not rewritten a second time, which is exactly why this terminates.

9: Frame -> False and Frame -> True as replacement rules

```
Clear[x];
Plot[Sin[x], {x, 0, 2 π}, Frame → False]

Plot[Sin[x], {x, 0, 2 π}, Frame → True]
```

Their purpose. Every option in Mathematica is a replacement rule, and that is not an analogy - option settings are literally Rule expressions carried in the argument sequence. Plot collects them, appends its own defaults, and resolves each option name against that list with ReplaceAll. Frame -> False draws the default crossed axes; Frame -> True draws a closed box with ticks on all four sides and moves the tick labels out to it. Because options are ordinary expressions, they can be manipulated by ordinary expression tools - which is the point of the next exercise.

10: Rewriting a finished plot by rewriting its options

```
Clear[x, plot1, plot2];
plot1 = Plot[Sin[x], {x, 0, 2  $\pi$ }, Frame  $\rightarrow$  True]

?plot1

Frame /. Options[plot1]

plot2 = plot1 /.
  (Frame  $\rightarrow$  {{True, True}, {True, True}})  $\rightarrow$  Frame  $\rightarrow$  {{False, True}, {False, True}};
Frame /. Options[plot2]

Show[plot2]
```

How it works. Plot does not store `Frame  $\rightarrow$  True` as written. It canonicalises the setting into the four-sided form `{{left, right}, {bottom, top}}`, so what is actually held inside the Graphics object is

`Frame  $\rightarrow$  {{True, True}, {True, True}}`. That is what `"?plot1"` shows, and it is why the replacement rule in this exercise has to be written against the canonical form rather than the form that was typed.

The rule replaces one whole option rule with another whole option rule - the pattern being matched is itself a Rule expression. The new setting turns off the left and bottom frame edges and keeps the right and top ones. Show re-renders the modified object.

This is the general moral of the lesson: a plot is not a picture, it is an expression, and every tool that works on expressions works on it.

11: DSolve, substituting constants, checking, and plotting

```
Clear[y, t, sol, sol2];
sol = DSolve[y''[t] + y[t] == 0, y[t], t]
```

Part 1. DSolve returns a list of lists of rules: the outer list holds one entry per solution branch, and here there is one branch, whose single rule sends `y[t]` to `C[1] Cos[t] + C[2] Sin[t]`. The arbitrary constants are `C[1]` and `C[2]` - subscripted C, not two separate symbols.

```
sol2 = sol /. {c1  $\rightarrow$  4, c2  $\rightarrow$  7}
```

Part 2. A list of two rules applied with `ReplaceAll` pins both constants, giving `y[t]  $\rightarrow$  4 Cos[t] + 7 Sin[t]`.

```
rule = First[sol2]
```

```
drule =  $\partial_{\{t,2\}}$  rule
```

```
Simplify[y''[t] + y[t] /. drule /. rule]
```

Part 3, and the hint. Substituting the solution back into the differential equation cannot be done with one `ReplaceAll`, because the rule's left-hand

side is $y[t]$ and the equation contains $y''[t]$, which is a different expression - $\text{Derivative}[2][y][t]$ - and does not match.

The hint asks what happens when a rule is differentiated. D threads over Rule , differentiating both sides, so $D[y[t] \rightarrow 4 \cos[t] + 7 \sin[t], \{t, 2\}]$ produces the new rule $y''[t] \rightarrow -4 \cos[t] - 7 \sin[t]$. That rule matches the second-derivative term.

Applying the differentiated rule first and the original rule second replaces both terms, and Simplify returns 0. The solution checks out.

```
ysol = y[t] /. rule
```

```
Plot[ysol, {t, 0, 2 \pi}]
```

Part 4. Extracting the solution from its rule is one more ReplaceAll : apply the rule to the expression $y[t]$ and what comes back is the right-hand side alone. The plot is a single sinusoid of amplitude $\sqrt{4^2 + 7^2} = \sqrt{65}$, which is the expected superposition of the two modes.